

Documentación Técnica

1. Descripción general del proyecto

AI-O HOME es una aplicación React + Vite + TypeScript para un marketplace de productos smart home. Tiene dos áreas principales:

- **Marketplace:** home, catálogo, detalle de producto, carrito/checkout y perfil.
- **Administración:** dashboard y pantalla de gestión de entidades.

La app se monta desde `src/main.tsx`, donde se configuran `BrowserRouter`, `AppProviders` y `AppRouter`. La navegación se define con React Router. El estado global se concentra en contextos de React para autenticación y tema. Los datos del prototipo se manejan con servicios mock, datos locales y `localStorage`.

También existe una carpeta `pages` con HTML estático legado/prototipo. La aplicación React actual está en `src`.

2. Estructura de carpetas

Carpeta	Responsabilidad
<code>src/app/layouts</code>	Layouts por área: auth, admin y marketplace.
<code>src/app/providers</code>	Composición de providers globales.
<code>src/app/router</code>	Definición de rutas y protección por autenticación/rol.
<code>src/components</code>	Componentes reutilizables por dominio y componentes base de UI.
<code>src/context</code>	Contextos globales de autenticación y tema.
<code>src/data/mock</code>	Datos iniciales mock de productos, carrito y perfil.
<code>src/hooks</code>	Hooks personalizados para lógica reutilizable.
<code>src/pages</code>	Páginas React principales.
<code>src/services</code>	Servicios de lógica mock y persistencia local.
<code>src/styles</code>	Estilos globales.
<code>src/types</code>	Tipos TypeScript del dominio.
<code>src/utils</code>	Helpers, formateadores y validadores simples.
<code>public</code>	Assets públicos.
<code>pages</code>	Prototipo HTML estático separado de la app React.
<code>dist</code>	Build generado.

3. Mapa de páginas

Página	Ruta	Función principal	Conexiones clave
--------	------	-------------------	------------------

Página	Ruta	Función principal	Conexiones clave
LoginPage.tsx	/auth/login	Login de usuario/admin.	AuthCard, LoginForm, useAuth.login; navega por rol.
RegisterPage.tsx	/auth/register	Registro simulado.	AuthCard, RegisterForm, useAuth.register; vuelve a login.
RecoverPage.tsx	/auth/recover	Recupero simulado de contraseña.	AuthCard, RecoverForm, useAuth.recover; muestra confirmación.
DashboardPage.tsx	/admin/dashboard	Métricas y accesos admin.	AdminStatCard, adminService.getStats; escucha cambios de records.
ListingsPage.tsx	/admin/listings?tab=...	CRUD visual de entidades admin.	Tabs, Button, useModal, adminService; maneja records, filtros, formularios y modales.
HomePage.tsx	/marketplace/home	Home del marketplace.	productService.list; envía productos a secciones como FeaturedProducts.
StorePage.tsx	/marketplace/store	Catálogo con búsqueda/filtros.	ProductCard, productService.list, marketplaceService; agrega productos al carrito.
ProductPage.tsx	/marketplace/product/:productId	Detalle de producto.	ProductDetail, productService.getById, marketplaceService; agrega al carrito.
CartPage.tsx	/marketplace/cart	Carrito y checkout.	useCheckoutFlow, CartItemCard, CartSidebar, CheckoutPanel, AddCardModal.
ProfilePage.tsx	/marketplace/profile	Perfil, historial y métodos de pago.	useAuth, useProfileDashboard, cards y modales de perfil.

4. Conexión entre componentes

Relación	Comunicación	Propósito
----------	--------------	-----------

Relación	Comunicación	Propósito
<code>main.tsx</code> → <code>AppProviders</code> → <code>AppRouter</code>	Providers envuelven las rutas.	Dar contexto global y navegación a toda la app.
<code>AppRouter</code> → <code>ProtectedRoute</code>	Recibe <code>role</code> opcional y <code>children</code> .	Validar sesión y rol antes de renderizar layouts privados.
<code>AdminLayout</code> → <code>Header</code> / <code>Outlet</code>	Pasa título, icono, acciones de tema/logout.	Encapsular navegación y acciones comunes admin.
<code>MarketplaceLayout</code> → <code>NavLink</code> / <code>Outlet</code>	Links principales y toggle de tema.	Encapsular navegación común del marketplace.
Páginas auth → forms	Los forms usan <code>useAuth</code> y <code>react-hook-form</code> .	Ejecutar login, registro o recupero y navegar según resultado.
<code>HomePage</code> → secciones marketplace	Pasa <code>products</code> .	Reutilizar datos de productos en hero, estadísticas y sliders.
<code>FeaturedProducts</code> → <code>ProductSlider</code>	Pasa productos agrupados y <code>onSelect</code> .	Navegar al detalle desde sliders.
<code>StorePage</code> → <code>ProductCard</code>	Pasa <code>product</code> y <code>onAddToCart</code> .	Renderizar catálogo y delegar evento de compra al padre.
<code>ProductPage</code> → <code>ProductDetail</code>	Pasa <code>product</code> y <code>onAddToCart</code> .	Mostrar detalle y delegar agregado al carrito.
<code>CartPage</code> → componentes de carrito	Pasa items, totales, métodos de pago y callbacks.	Concentrar la lógica en <code>useCheckoutFlow</code> y renderizar cada bloque.
<code>ProfilePage</code> → cards/modales de perfil	Pasa perfil, órdenes, métodos y callbacks.	Mostrar datos y pedir confirmación para acciones sensibles.
<code>DashboardPage</code> → <code>AdminStatCard</code>	Pasa label, valor y tab.	Entrar a listados filtrados por módulo.
<code>ListingsPage</code> → UI interna	Maneja estado y callbacks localmente.	Centralizar edición, borrado, filtros, relaciones e imágenes.

5. Routing y navegación

Aspecto	Detalle
Librería	<code>react-router-dom</code> v6.
Archivo principal	<code>src/app/router/AppRouter.tsx</code> .
Router base	<code>BrowserRouter</code> en <code>src/main.tsx</code> .
Rutas públicas	<code>/auth/login</code> , <code>/auth/register</code> , <code>/auth/recover</code> .

Aspecto	Detalle
Rutas marketplace	<code>/marketplace/home</code> , <code>/marketplace/store</code> , <code>/marketplace/product/:productId</code> , <code>/marketplace/cart</code> , <code>/marketplace/profile</code> .
Rutas admin	<code>/admin/dashboard</code> , <code>/admin/listings</code> .
Redirecciones	Ruta desconocida a <code>/auth/login</code> ; usuario sin sesión a login; usuario sin rol admin a <code>/marketplace/home</code> .
Parámetros	<code>productId</code> en detalle de producto; <code>tab</code> por query string en listados admin.
Navegación	<code>Link</code> , <code>NavLink</code> , <code>useNavigate</code> y <code>Navigate</code> .

6. Estado de la aplicación

Tipo de estado	Uso
<code>useState</code>	Formularios simples, productos, filtros, carrito, checkout, perfil, modales y records admin.
<code>useEffect</code>	Carga inicial, sincronización con storage, listeners de eventos y scroll top.
<code>useMemo</code>	Productos filtrados, totales de compra, agrupaciones y datos derivados del perfil.
<code>useContext</code>	Acceso a autenticación y tema mediante hooks propios.
<code>react-hook-form</code>	Formularios de auth y alta de tarjeta.
<code>localStorage</code>	Usuario, rol, tema, carrito, perfil y records admin.

Estado global:

- `AuthContext`: usuario actual y acciones de auth.
- `ThemeContext`: tema actual y alternancia claro/oscuro.

Claves persistidas principales:

- `aio-auth-user`
- `role`
- `aio-theme`
- `aio-marketplace-cart`
- `aio-marketplace-profile`
- `aio-admin-records`

7. Servicios y métodos

Servicio	Métodos principales	Usado por	Responsabilidad
<code>auth.service.ts</code>	<code>login</code> , <code>register</code> , <code>recover</code> , <code>logout</code> , <code>getCurrentUser</code>	<code>AuthContext</code> , páginas auth, layouts, perfil	Autenticación mock, persistencia de usuario/rol y cierre de sesión.

Servicio	Métodos principales	Usado por	Responsabilidad
<code>admin.service.ts</code>	<code>getRecordsByTabSync,</code> <code>saveRecordsByTab,</code> <code>subscribeToRecordsChanges,</code> <code>getStats, listByTab</code>	<code>DashboardPage,</code> <code>ListingsPage</code>	Lectura, guardado y sincronización de records admin.
<code>marketplace.service.ts</code>	<code>getInitialCartItems,</code> <code>setCartItems, clearCart,</code> <code>getProfile, setProfile,</code> <code>appendOrder,</code> <code>appendProfilePaymentMethod,</code> <code>removeProfilePaymentMethod,</code> <code>onStateChange,</code> <code>createOrderFromCart</code>	<code>StorePage,</code> <code>ProductPage,</code> <code>useCheckoutFlow,</code> <code>useProfileDashboard</code>	Carrito, perfil, órdenes, métodos de pago y eventos de sincronización.
<code>product.service.ts</code>	<code>list, getById</code>	<code>HomePage, StorePage,</code> <code>ProductPage</code>	Consulta de productos mock.
<code>storage.service.ts</code>	<code>get, set, remove</code>	<code>auth.service.ts,</code> <code>admin.service.ts</code>	Wrapper de <code>localStorage</code> con fallback ante errores de parseo.

8. Hooks personalizados

Hook	Archivo	Uso principal
<code>useAuth</code>	<code>src/hooks/useAuth.ts</code>	Acceder al contexto de autenticación desde rutas, formularios, layouts y perfil.
<code>useTheme</code>	<code>src/hooks/useTheme.ts</code>	Acceder al tema y alternarlo desde layouts.
<code>useLocalStorage</code>	<code>src/hooks/useLocalStorage.ts</code>	Helper genérico de storage. No se identificó uso activo.
<code>useModal</code>	<code>src/hooks/useModal.ts</code>	Abrir/cerrar modales y guardar payload opcional.
<code>useCheckoutFlow</code>	<code>src/hooks/useCheckoutFlow.ts</code>	Manejar items, totales, métodos de pago, compra y éxito del checkout.
<code>useProfileDashboard</code>	<code>src/hooks/useProfileDashboard.ts</code>	Manejar perfil, orden seleccionada, métodos de pago y totales de historial.

9. Validaciones y manejo de errores

Área	Validaciones / errores
Auth login	Campos requeridos y error visible si las credenciales mock no coinciden.
Register	Campos principales requeridos; la acción es simulada.

Área	Validaciones / errores
Recover	Email requerido y mensaje de confirmación.
Store	Filtros locales por texto, categoría, asistente, conectividad y precio máximo.
Product detail	Si no encuentra producto, muestra Product not found .
Cart	No permite avanzar a checkout sin items; cantidad en cero elimina el item.
Checkout	Requiere método de pago seleccionado; el modal de tarjeta valida número, vencimiento y CVV.
Profile	Usa modales de confirmación para editar perfil y borrar métodos de pago.
Admin listings	Valida relaciones, reglas de promos, precio/stock y errores de campos del formulario.
Storage	Los servicios devuelven fallback ante JSON inválido.

10. Dependencias relevantes

Dependencia	Uso
react	Componentes, hooks y estado.
react-dom	Montaje de la app.
react-router-dom	Routing, links y navegación programática.
react-hook-form	Formularios y validaciones.
vite	Dev server y build.
typescript	Tipado estático.
tailwindcss, postcss, autoprefixer	Estilos utilitarios y procesamiento CSS.
@vitejs/plugin-react	Integración React con Vite.

11. Observaciones técnicas

- `src/pages/admin/ListingsPage.tsx` concentra mucha lógica: campos, datos iniciales, validaciones, formularios, modales, imágenes y order items. Conviene separarlo por responsabilidad si el módulo crece.
- Existen componentes admin reutilizables (`AdminRecordCard`, `AdminSearchBar`, `AdminFilters`, `AdminTable`, `AdminCrudModal`) que no se identificaron como usados por `ListingsPage`.
- `authService.register` y `authService.recover` son simulados y no persisten usuarios nuevos.
- `StorePage` y `ProductPage` duplican la lógica de agregar productos al carrito. Conviene extraerla si se agregan más puntos de compra.
- `CheckoutPanel` mantiene datos de facturación locales, pero la orden creada no incluye esos datos.
- Hay algunos textos renderizados con problemas de codificación en componentes existentes. Conviene revisarlos antes de una entrega final.
- La carpeta `pages` HTML y el README describen una versión estática anterior. Para evitar confusión, conviene aclarar que `src` es la app React vigente.